OBJECT ORIENTED PROGRAMMING
CCS20704, DCS20504, CCS23803

Received Date : 3 September 2026
Submission Due Date: 23 October 2026
Lecturer : Dr. Jamal Abdullahi
Weightage : 15%
Semester : SEPTEMBER 2026

Instruction to students:

- This is a GROUP assignment.
- Complete this cover sheet and attach it to your assignment (first page).

Student declaration:		
I declare that: • ***This assignment is my/our own work.*** • ***I/we understand what is meant by plagiarism.*** • ***My lecturer has the right to deduct my marks in case of:*** - ***Late submission*** - ***Any plagiarism found in my assignment.***		
NAME: MUHAMMAD IFTIKHAR BIN SANUSI AFKAR **MATRICS NO: 012026070955** 	**NAME: MUHAMMAD RAFIQ FIRDAUS RAFHAN** **MATRICS NO: 012026070899** 	**NAME:** **MATRICS NO:** YOUR PHOTO

1.0 Introduction

For this assignment, we developed a car rental system as the chosen topic. The system allows users to view car details, calculate rental costs based on the number of rental days, and check whether a car requires servicing based on its mileage. It also provides information such as the car ID, brand, model, plate number, daily rental rate, mileage, and availability status.

2.0 System Design

2.1 Class and Object

```
RentalCar.java > ...
1   public class RentalCar {
2
```

```
3
4   public class Main {
5
```

```
8
9       // Creating Object 1 using Default Constructor
10      RentalCar car1 = new RentalCar();
11
12      // Creating Object 2 using Parameterized Constructor
13      RentalCar car2 = new RentalCar("C002", "McLaren", "Myvi", "SRG 1010", 150.00, 15000, true);
14
```

Figure 1, 2 & 3. Classes and Object of RentalCar.java and Main.java

RentalCar is the class blueprint. car1 and car2 are separate objects. car1 uses the default constructor and represents a Lamborghini Axia; car2 uses the parameterized constructor and represents a Mclaren Myvi.

2.2 Attributes

```
3       // List of attributes
4       private String carId;
5       private String brand;
6       private String model;
7       private String plateNumber;
8       private double dailyRate;
9       private int mileage;
10      private boolean isAvailable;
```

Figure 4. Declaration of Private Attributes in RentalCar.java

Attribute	Type	Purpose
carId	String	As car identifier, exp is C001
brand	String	Manufacturer.
model	String	Model name.
plateNumber	String	Registration number.
dailyRate	double	Rental price per day in RM.
mileage	int	Distance in kilometres.
isAvailable	boolean	true if available and false if rented.

Table 1

2.3 Constructors

```java
    // Default Constructor
    public RentalCar() {
        this.carId = "C001";
        this.brand = "Lamborghini";
        this.model = "Axia";
        this.plateNumber = "MLK 0404";
        this.dailyRate = 100.00;
        this.mileage = 10000;
        this.isAvailable = true;
    }

    // Parameterized Constructor
    public RentalCar(String carId, String brand, String model, String plateNumber,
                     double dailyRate, int mileage, boolean isAvailable) {
        this.carId = carId;
        this.brand = brand;
        this.model = model;
        this.plateNumber = plateNumber;
        this.dailyRate = dailyRate;
        this.mileage = mileage;
        this.isAvailable = isAvailable;
    }
```

Figure 5 & 6. Default and Parameterized Constructor in RentalCar.java

The default constructor initializes a Lamborghini Axia with ID C001, plate MLK 0404, RM100 daily rate, 10,000 km mileage, and available status. The parameterized constructor accepts values for all seven attributes make it fully parameterized constructor.

2.4 Methods

```java
    // Setter and Getter methods for each attribute
    public String getCarId() {
        return carId;
    }

    public void setCarId(String carId) {
        this.carId = carId;
    }

    public String getBrand() {
        return brand;
    }

    public void setBrand(String brand) {
        this.brand = brand;
    }
```

```java

    public String getModel() {
        return model;
    }

    public void setModel(String model) {
        this.model = model;
    }

    public String getPlateNumber() {
        return plateNumber;
    }

    public void setPlateNumber(String plateNumber) {
        this.plateNumber = plateNumber;
    }

    public double getDailyRate() {
        return dailyRate;
    }
```

```java
    // Setter method with validation to ensure daily rate is more than RM 0
    public void setDailyRate(double dailyRate) {
        if (dailyRate > 0) {
            this.dailyRate = dailyRate;
        } else {
            // Displaying message if the daily rate is invalid
            System.out.println("Invalid rate! Rate must be more than RM 0.");
        }
    }

    // Getter and Setter methods for mileage and availability
    public int getMileage() {
        return mileage;
    }

    public void setMileage(int mileage) {
        this.mileage = mileage;
    }

    public boolean getIsAvailable() {
        return isAvailable;
    }

    public void setIsAvailable(boolean isAvailable) {
        this.isAvailable = isAvailable;
    }
```

```java
    // Additional method to display car information
    public void displayCarInfo() {
        System.out.println("-----------------------------------");
        System.out.println("Car ID       : " + carId);
        System.out.println("Brand & Model: " + brand + " " + model);
        System.out.println("Plate Number : " + plateNumber);
        System.out.println("Daily Rate   : RM " + dailyRate);
        System.out.println("Mileage      : " + mileage + " km");
        if (isAvailable) {
            System.out.println("Status       : Available for Rent");
        } else {
            System.out.println("Status       : Currently Rented Out");
        }
        System.out.println("-----------------------------------");
    }

    // Additional method to calculate total rental cost based on the number of days rented
    public double calculateTotalCost(int days) {
        double total = dailyRate * days;
        return total;
    }

    // Additional method to check the service status of the car based on mileage
    public void checkServiceStatus() {
        if (mileage >= 50000) {
            System.out.println("Maintenance Alert: " + brand + " " + model + " needs servicing (Mileage: " + mileage + " km).");
        } else {
            // Displaying message if the car is in good condition
            System.out.println("Maintenance Status: " + brand + " " + model + " is in good condition.");
        }
```

Figure 7, 8, 9 & 10. Methods

Method	Purpose
displayCarInfo()	Displays attributes and availability.
calculateTotalCost(int days)	Returns daily rate multiplied by rental days.
checkServiceStatus()	Alerts when mileage is at least 50,000 km.
Getters	Return attribute values.
Setters	Modify attributes; setDailyRate validates positive rates.

Table 2

2.5 Object Oriented Programming Concept

i. Encapsulation

```
3     // List of attributes
4     private String carId;
5     private String brand;
6     private String model;
7     private String plateNumber;
8     private double dailyRate;
9     private int mileage;
10    private boolean isAvailable;
```

Figure 11. Encapsulation Through Private Attributes

Encapsulation combines data and methods in a class while controlling access. All seven RentalCar attributes are private, so Main uses public getters and setters.

ii. Abstraction

```
116     // Additional method to calculate total rental cost based on the number of days rented
117     public double calculateTotalCost(int days) {
118         double total = dailyRate * days;
119         return total;
120     }
```

```
122     // Additional method to check the service status of the car based on mileage
123     public void checkServiceStatus() {
124         if (mileage >= 50000) {
125             System.out.println("Maintenance Alert: " + brand + " " + model + " needs servicing (Mileage: " + mileage + " km).");
126         } else {
127             // Displaying message if the car is in good condition
128             System.out.println("Maintenance Status: " + brand + " " + model + " is in good condition.");
129         }
```

Figure 12 & 13. Implementation of Abstraction Using Operational Methods

Methods hide implementation details. Main calls calculateTotalCost(days) instead of repeating the multiplication formula, and checkServiceStatus() contains the mileage comparison.

2.6 The Program Control Structure

1. While loop

```
22          // Looping structure: using while loop to repeat the menu until user enters 6
23          while (choice != 6) {
24              System.out.println("\nMAIN MENU:");
25              System.out.println("1. View Car Details");
26              System.out.println("2. Rent a Car & Calculate Price");
27              System.out.println("3. Return a Car");
28              System.out.println("4. Update Car Daily Rate");
29              System.out.println("5. Check Car Service Status");
30              System.out.println("6. Exit");
31              System.out.print("Enter your choice (1-6): ");
32              choice = input.nextInt();
```

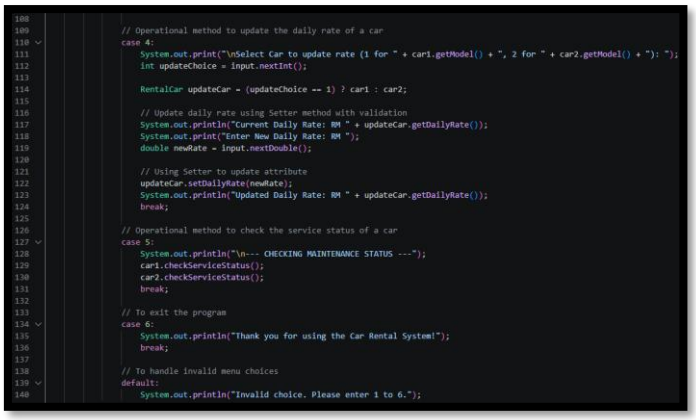
Figure 14. While Loop for Menu

The while loop repeats the menu until the user selects option 6.

2. switch statement

```
34          // Selection structure: using switch statement for menu selection
35          switch (choice) {
36
37              // Operational method to display car information
38              case 1:
39                  System.out.println("\n--- CAR 1 DETAILS ---");
40                  car1.displayCarInfo();
41                  System.out.println("\n--- CAR 2 DETAILS ---");
42                  car2.displayCarInfo();
43                  break;
44
45              // Operational method to rent a car and calculate total cost
46              case 2:
47                  System.out.print("\nSelect Car to rent (1 for " + car1.getModel() + ", 2 for " + car2.getModel() + "): ");
48                  int rentChoice = input.nextInt();
49
50                  RentalCar selectedCar;
51                  if (rentChoice == 1) {
52                      selectedCar = car1;
53                  } else {
54                      selectedCar = car2;
55                  }
56
57                  // Selection structure: usingif-else to check if the car is available
58                  if (selectedCar.getIsAvailable()) {
59                      System.out.print("Enter number of days to rent: ");
60                      int days = input.nextInt();
61
62                      // Looping structure: usingfor loop to display day by day base rate breakdown
63                      System.out.println("\n--- Daily Breakdown ---");
64                      for (int day = 1; day <= days; day++) {
65                          System.out.println("Day " + day + ": RM " + selectedCar.getDailyRate());
66                      }
67
68                      // Operational method to calculate total cost
69                      double totalCost = selectedCar.calculateTotalCost(days);
70
71                      // Modify availability attribute using Setter
72                      selectedCar.setIsAvailable(false);
73
74                      // Displaying the total cost to pay
75                      System.out.println("\n--- TOTAL COST ---");
76                      System.out.println("Total Price to Pay: RM " + totalCost);
77                      System.out.println("Success! You have rented the " + selectedCar.getBrand() + " " + selectedCar.getModel() + ".");
78                  } else {
```

```
79                      // Displaying message if the car is not available for rent
80                      System.out.println("Sorry! That car is currently rented out.");
81                  }
82                  break;
83
84              // Operational method to return a car and update mileage
85              case 3:
86                  System.out.print("\nSelect Car to return (1 for " + car1.getModel() + ", 2 for " + car2.getModel() + "): ");
87                  int returnChoice = input.nextInt();
88
89                  RentalCar returnCar = (returnChoice == 1) ? car1 : car2;
90
91                  // Selection structure: using if-else to check if the car is currently rented out
92                  if (!returnCar.getIsAvailable()) {
93                      System.out.print("Enter distance driven (km): ");
94                      int drivenKm = input.nextInt();
95
96                      // Manipulating attributes using Getters and Setters
97                      int newMileage = returnCar.getMileage() + drivenKm;
98                      returnCar.setMileage(newMileage);
99                      returnCar.setIsAvailable(true);
100
101                     // Displaying message after returning the car successfully
102                     System.out.println("Car returned successfully! Updated Mileage: " + returnCar.getMileage() + " km.");
103                 } else {
104                     // Displaying message if the car is already available in the garage
105                     System.out.println("This car is already available in the garage.");
106                 }
107                 break;
108
109             // Operational method to update the daily rate of a car
110             case 4:
111                 System.out.print("\nSelect Car to update rate (1 for " + car1.getModel() + ", 2 for " + car2.getModel() + "): ");
112                 int updateChoice = input.nextInt();
113
114                 RentalCar updateCar = (updateChoice == 1) ? car1 : car2;
115
116                 // Update daily rate using Setter method with validation
117                 System.out.println("Current Daily Rate: RM " + updateCar.getDailyRate());
118                 System.out.print("Enter New Daily Rate: RM ");
119                 double newRate = input.nextDouble();
```



```
// Operational method to update the daily rate of a car
case 4:
    System.out.print("\nSelect Car to update rate (1 for " + car1.getModel() + ", 2 for " + car2.getModel() + "): ");
    int updateChoice = input.nextInt();

    RentalCar updateCar = (updateChoice == 1) ? car1 : car2;

    // Update daily rate using Setter method with validation
    System.out.println("Current Daily Rate: RM " + updateCar.getDailyRate());
    System.out.print("Enter New Daily Rate: RM ");
    double newRate = input.nextDouble();

    // Using Setter to update attribute
    updateCar.setDailyRate(newRate);
    System.out.println("Updated Daily Rate: RM " + updateCar.getDailyRate());
    break;

// Operational method to check the service status of a car
case 5:
    System.out.println("\n--- CHECKING MAINTENANCE STATUS ---");
    car1.checkServiceStatus();
    car2.checkServiceStatus();
    break;

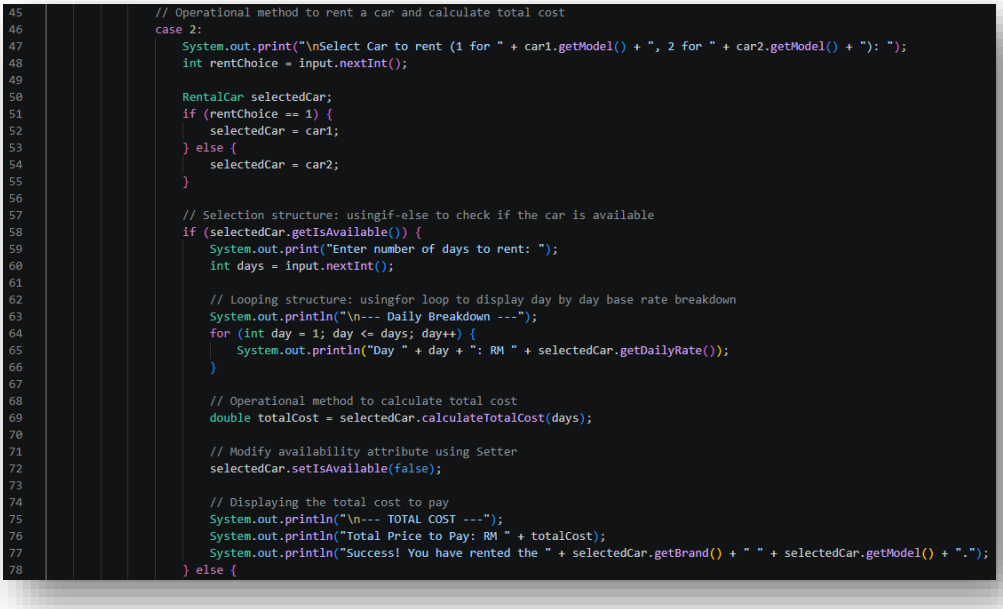
// To exit the program
case 6:
    System.out.println("Thank you for using the Car Rental System!");
    break;

// To handle invalid menu choices
default:
    System.out.println("Invalid choice. Please enter 1 to 6.");
```

Figure 15, 16 & 18. Switch Statement for Main Menu

The switch statement routes menu choices to view details, rent, return, update the rate, check service status, or exit. The default branch reports invalid choices.

3. if-else selection



```
// Operational method to rent a car and calculate total cost
case 2:
    System.out.print("\nSelect Car to rent (1 for " + car1.getModel() + ", 2 for " + car2.getModel() + "): ");
    int rentChoice = input.nextInt();

    RentalCar selectedCar;
    if (rentChoice == 1) {
        selectedCar = car1;
    } else {
        selectedCar = car2;
    }

    // Selection structure: usingif-else to check if the car is available
    if (selectedCar.getIsAvailable()) {
        System.out.print("Enter number of days to rent: ");
        int days = input.nextInt();

        // Looping structure: usingfor loop to display day by day base rate breakdown
        System.out.println("\n--- Daily Breakdown ---");
        for (int day = 1; day <= days; day++) {
            System.out.println("Day " + day + ": RM " + selectedCar.getDailyRate());
        }

        // Operational method to calculate total cost
        double totalCost = selectedCar.calculateTotalCost(days);

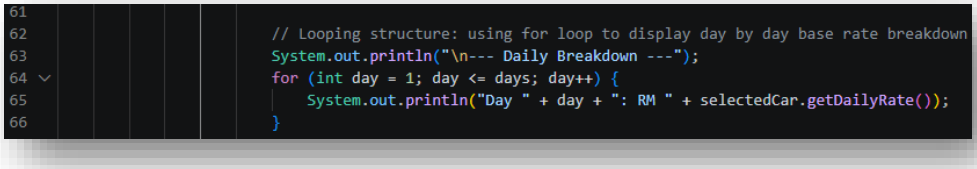
        // Modify availability attribute using Setter
        selectedCar.setIsAvailable(false);

        // Displaying the total cost to pay
        System.out.println("\n--- TOTAL COST ---");
        System.out.println("Total Price to Pay: RM " + totalCost);
        System.out.println("Success! You have rented the " + selectedCar.getBrand() + " " + selectedCar.getModel() + ".");
    } else {
```

Figure 19. If-Else Statement for Checking Car Availability

The rent option checks whether the car is available. If available, the program calculates the price and marks the car unavailable. Otherwise, it displays a message. Similar conditions are used when returning a car and checking service status.

4. for loop



```
// Looping structure: using for loop to display day by day base rate breakdown
System.out.println("\n--- Daily Breakdown ---");
for (int day = 1; day <= days; day++) {
    System.out.println("Day " + day + ": RM " + selectedCar.getDailyRate());
}
```

Figure 20. For Loop for Displaying the Daily Rental Breakdown

The for loop prints the daily rental rate for each day entered by the user.

5. getter and setter usage

```
96                  // Manipulating attributes using Getters and Setters
97                  int newMileage = returnCar.getMileage() + drivenKm;
98                  returnCar.setMileage(newMileage);
99                  returnCar.setIsAvailable(true);
```

```
116                 // Update daily rate using Setter method with validation
117                 System.out.println("Current Daily Rate: RM " + updateCar.getDailyRate());
118                 System.out.print("Enter New Daily Rate: RM ");
119                 double newRate = input.nextDouble();
120
121                 // Using Setter to update attribute
122                 updateCar.setDailyRate(newRate);
123                 System.out.println("Updated Daily Rate: RM " + updateCar.getDailyRate());
124                 break;
```

Figure 21 & 22. Getter and Setter Usage for Updating Car Attributes

Getters read details such as model, rate, mileage, and availability. Setters update availability, mileage, and daily rate.

2.7 Output

1. Main Menu

```
========================================
          CAR RENTAL SYSTEM
========================================

MAIN MENU:
1. View Car Details
2. Rent a Car & Calculate Price
3. Return a Car
4. Update Car Daily Rate
5. Check Car Service Status
6. Exit
Enter your choice (1-6): 1
```

Figure 23. Main Menu of the Car Rental System

Main Menu of the Car Rental System, displaying all of the available option 1 – 6.

2. Car Details

```
--- CAR 1 DETAILS ---
-----------------------------------
Car ID       : C001
Brand & Model: Lamborghini Axia
Plate Number : MLK 0404
Daily Rate   : RM 100.0
Mileage      : 10000 km
Status       : Available for Rent
-----------------------------------

--- CAR 2 DETAILS ---
-----------------------------------
Car ID       : C002
Brand & Model: McLaren Myvi
Plate Number : SRG 1010
Daily Rate   : RM 150.0
Mileage      : 15000 km
Status       : Available for Rent
-----------------------------------
```

Figure 24. Displaying Car Details

Shows the output of option 1, displaying the Car ID, brand, model, registration number, daily rental rate, mileage, and availability of both cars.

3. Renting a Car

```
MAIN MENU:
1. View Car Details
2. Rent a Car & Calculate Price
3. Return a Car
4. Update Car Daily Rate
5. Check Car Service Status
6. Exit
Enter your choice (1-6): 2

Select Car to rent (1 for Axia, 2 for Myvi): 1
Enter number of days to rent: 3

--- Daily Breakdown ---
Day 1: RM 100.0
Day 2: RM 100.0
Day 3: RM 100.0

--- TOTAL COST ---
Total Price to Pay: RM 300.0
Success! You have rented the Lamborghini Axia.
```

Figure 25. Renting a Car and Calculating the Rental Cost

Shows the selected vehicle, the daily rental breakdown, the total price, and the confirmation message after a successful rental.

4. Returning the Car

```
MAIN MENU:
1. View Car Details
2. Rent a Car & Calculate Price
3. Return a Car
4. Update Car Daily Rate
5. Check Car Service Status
6. Exit
Enter your choice (1-6): 3

Select Car to return (1 for Axia, 2 for Myvi): 1
Enter distance driven (km): 200
Car returned successfully! Updated Mileage: 10200 km.
```

Figure 26. Returning the Car and Updating its Mileage

Shows how the distance driven is added to the existing mileage and changes the car's availability back to true.

5. **Updating the Rate of a Car**

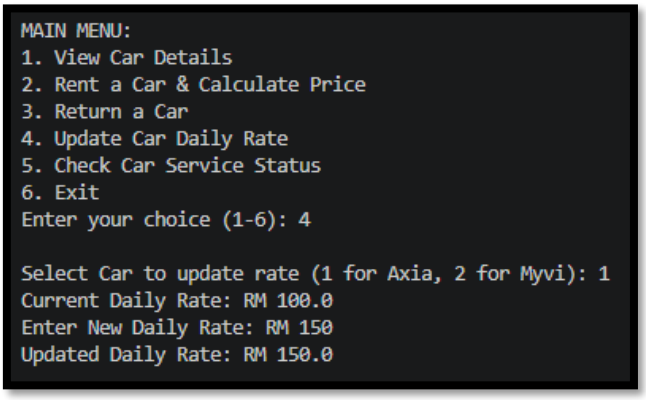


Figure 27. Updating the Car's Daily Rental Rate

Shows the how to update the daily rate in the system.

6. **Check Car Service Status**

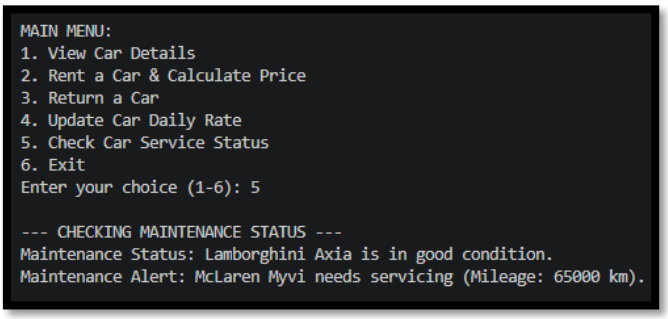


Figure 28. Check Car Service Status

Shows the maintenance status of both vehicles based on their mileage.

7. **End of Program**

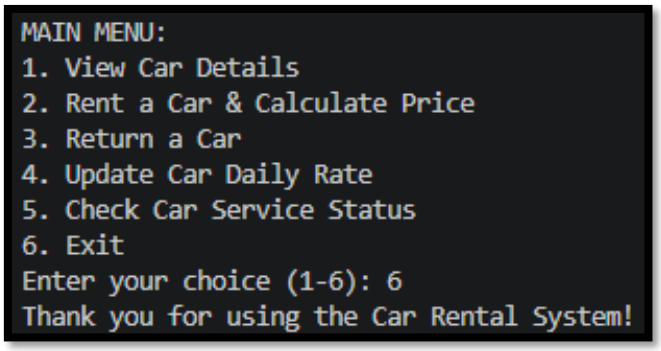


Figure 29. Ending the Program

Shows how to end the program.

3.0 Appendix

GitHub Repository Link: https://github.com/iftikharsanusi/Assigment-1-Object-Oriented-Programming

Iftikhar: Main.java
Rafiq: RentalCar.java